ECEN 5733 miniproject 1 AI usage appendix:

(Note, a transcript is included in the appropriate jupyter notebook at the pertinent section).

Prompt:

```
can you create a function that plots the decision boundries given the weights and biases? #define base perceptron class
class Perceptron():
'''single layer perceptron class. From textbook inputs p should have n_inputs=R and the output of the model should have n_outputs=S'''
def __init__(self, weight_shape:list|None=None, weights:np.ndarray=None, bias:np.ndarray=None, activation=hardlims):
self.activation=activation
if weights is None:
if weight_shape is None: #weight shape should be [S,R]
raise ValueError('you must provide a weight or input/output shape')
weights=np.random.rand(*weight_shape)
self.W=np.asarray(weights).astype(float) #ensure float type

#set shape vars and ensure 2d
if self.W.ndim==1: self.W=self.W.reshape(1,-1)
self.S=self.W.shape[0]
self.R=self.W.shape[1]

#ensure b is floats with 2 dimensions
if bias is None:
bias=np.random.rand(*[1,self.S])
self.b=np.asarray(bias).astype(float)
if self.b.ndim==1: self.b=self.b.reshape(1,self.S)

def _shape_input(self, p:np.ndarray)->np.ndarray:
p=np.asarray(p)
if p.ndim == 1:
p = p.reshape(1, -1)
elif p.ndim != 2:
raise ValueError(f"Input must be 1D or 2D array, got {p.ndim}D.")
assert p.shape[1]==self.R , f'input shape {p.shape} doesn\'t match expected length {self.R}'
return p
def forward(self, p:np.ndarray)->np.ndarray:
'''compute forward pass through the perception for input vector p'''
#safety check
p=self._shape_input(p)
#compute output
```

```
n=p@self.W.T
n=n+self.b
return self.activation(n).reshape(-1,self.S)
```

Response:

```
import numpy as np
import matplotlib.pyplot as plt

def plot_decision_boundaries(perceptron, xlim=(-10, 10), ylim=(-10, 10),
ax=None, labels=None):
"""
Plot decision boundaries for a 2-input Perceptron.

Each output neuron defines:
w1*x + w2*y + b = 0

Parameters
----------
perceptron : Perceptron
Perceptron whose weights and biases should be plotted.
xlim, ylim : tuple
Plotting limits for x and y.
ax : matplotlib.axes.Axes, optional
Existing axes to plot on.
labels : list, optional
Labels for each decision boundary.
"""

if perceptron.R != 2:
raise ValueError(
f"Decision boundary plotting requires exactly 2 inputs, "
f"but this perceptron has R={perceptron.R}."
)

if ax is None:
fig, ax = plt.subplots(figsize=(8, 6))

x = np.linspace(xlim[0], xlim[1], 500)

for i in range(perceptron.S):
w1, w2 = perceptron.W[i]
```

```
b = perceptron.b[0, i]

# w1*x + w2*y + b = 0
# y = -(w1*x + b) / w2
if np.isclose(w2, 0):
# Vertical boundary: x = -b/w1
if not np.isclose(w1, 0):
x_boundary = -b / w1
ax.axvline(
x_boundary,
label=labels[i] if labels else f"Boundary {i+1}"
)
else:
y = -(w1 * x + b) / w2
ax.plot(
x, y,
label=labels[i] if labels else f"Boundary {i+1}"
)

ax.set_xlim(xlim)
ax.set_ylim(ylim)
ax.set_xlabel("$x_1$")
ax.set_ylabel("$x_2$")
ax.set_title("Perceptron Decision Boundaries")
ax.axhline(0, color="black", linewidth=0.5)
ax.axvline(0, color="black", linewidth=0.5)
ax.grid(True, alpha=0.3)
ax.legend()

return ax
```

The generated function is verbose and doesn't take the input vectors or labels into account. I modified the function heavily as indicated in section 5 of the notebook. Specifically, I made axis limits dynamic and added a scatter plot of input vectors.